

Bridging Generative AI and Optimization

A Collaborative Planning Approach to Solving the Traveling Salesman Problem

Course : GEN-AI and LLM

Instructor : Prof. Nidhi Goyal

Date : 28th-April-2026

Sreeja Moganty	SE23UECM056
Hemanth Varma	SE23UARI062
Pradyun Jakkidi	SE23UCSE079

Abstract

The combination of Generative Artificial Intelligence and traditional optimization algorithms has been demonstrated to be a productive way forward for solving optimization problems in planning and decision making scenarios. The present paper describes a hybrid algorithm where the reasoning capacity of a large language model (LLM) can be integrated with the accuracy and efficiency of numerical optimization. The purpose of this study is to investigate the ways a generative model can facilitate solving optimization problems while taking into account the drawbacks of such models in terms of constraint satisfaction and finding optimal solutions.

In order to measure the performance of the proposed algorithm, a problem of computational complexity will be used as the test case. The Traveling Salesman Problem (TSP) will serve as the optimization task in our framework, since it is highly relevant and computationally complex. Within this framework, an initial solution to the problem will be provided by a Large Language Model as an output to a natural language problem prompt. The quality of this output will be assessed against the output of a popular optimization framework such as Google OR-Tools.

As evidenced from the experimental results on several TSP instances, although LLMs are able to provide valid solutions to a problem, such solutions tend to be far from optimal when compared to the results obtained from classic approaches to solving optimization problems. In particular, OR-Tools outperforms the other model with regard to obtaining the shortest distance in TSP. Nevertheless, LLMs can be said to have an edge with regard to their flexibility and adaptive capacity.

Based on the results of this investigation, the integration of LLM-based inference models with the optimization procedures is highly promising, and could serve as a powerful means of bridging the gap between flexibility and accuracy in the process. This strategy holds great promise for practical use cases. Future work may focus on improving the current solution by introducing iterative refinement mechanisms and implementing scalable architecture based on high-performance computing facilities.

Table of Contents

1. Introduction	4
2. Literature Review	4
2.1 Generative AI for Decision Making	4
2.2 LLMs for Optimization	4
2.3 Limitations of LLM-Based Optimization.....	5
2.4 Gap in Existing Approaches	5
3. Problem Statement	5
4. Methodology	6
4.1 Overview	6
4.2 System Architecture	6
4.3 Key Components	7
4.3.1 LLM-Based Planner	7
4.3.2 Optimization Engine	7
4.3.3 Integration Layer	8
4.4 Workflow Description	8
4.5 Proposed Algorithm	8
4.6 Key Advantages	8
4.6.1 Performance & Accuracy	8
4.6.2 Flexibility & Usability	9
4.6.3 Reliability & Constraints	9
5. Experimental Setup	9
5.1 Problem Definition	9
5.2 Dataset	10
5.3 Prompting Strategy	10
5.4 Implementation	10
5.5 Experimental Design	11
5.6 Execution Environment	11
6. Results and Analysis	12
6.1 Results Table	12
6.2 Observations	13
6.3 Analysis	13
6.4 Key Insights	14
7. Conclusion and Future Work	15
7.1 Conclusion	15
7.2 Future Work	15
8. References	15
9. Appendix A: Code Access	15

1. Introduction

Generative Artificial Intelligence is getting better and better with the creation of Large Language Models. These models let machines do complicated things like think and solve problems. They are really good at a lot of things, such as understanding what people say, making plans and helping with decisions.

However, despite their strengths, LLMs are not very good at solving problems that have a lot of rules and restrictions. They often cannot find solutions that work and are not sure if their answers are the ones. On the other hand, classical optimization techniques, such as those implemented in tools like Google OR-Tools, are highly effective at solving constrained problems but lack flexibility and intuitive reasoning capabilities.

This results in a gap between generative reasoning and mathematical optimization, as each of these is insufficient on its own to solve complex planning problems in the real world. In an attempt to resolve this issue, we propose here a novel paradigm that integrates both the power of language models in reasoning with mathematical optimization.

The main focus of our work is the development and assessment of a cooperative planning strategy whereby language models create a plan that is then improved upon and optimized by constraint optimization.

2. Literature Review

2.1 Generative AI for Decision Making

The field of Automated Reasoning and Decision Making has seen tremendous progress in recent times owing to Generative Artificial Intelligence technologies such as Large Language Models. These AI models can understand natural language queries and provide structured outputs, making them useful in solving planning and decision problems. The capability to handle unstructured data makes them especially relevant for practical situations where the problem statements themselves are ambiguous.

2.2 LLMs for Optimization

Generative Artificial Intelligence, Specifically Large Language Models
Recent studies have investigated the ability of large language models to optimize. Among these is "Large Language Models as Optimizers," which proposes an Optimization by PROMPTing (OPRO) method.

This technique involves presenting the problem in natural language and having the LLM propose solutions through an iterative process. The generated solution proposals are scored, and the scores are used to guide the next generation of better solutions. As such, the process enables the model to keep improving its solutions without resorting to any mathematical modeling.

According to the research, LLMs are capable of generating solutions for various optimization problems including linear regressions and combinatorial problems. Furthermore, the method yields improvement in prompt optimization processes as evidenced by more accurate solutions than human-generated ones.

2.3 Limitations of LLM-Based Optimization

However, there are some drawbacks in applying LLMs for optimizer purposes.

First of all, they can give wrong or erroneous results, specifically when high numerical accuracy is important. For instance, LLMs may provide incorrect function values or even false solutions. Moreover, LLMs can repeat previous solutions or be incapable of understanding the tasks, thus making the exploration less efficient.

Also, LLMs have trouble breaking out of local solutions and optimizing on complicated surfaces.

Lastly, they do not guarantee feasibility and optimal solutions in the face of hard constraints.

2.4 Gap in Existing Approaches

It can be observed from the above discussion that current methodologies depend only on one of the two categories discussed above.

Language Model-based methodologies allow for flexibility and reasoning capabilities but are not reliable and fail to impose constraints.

Optimization-based methods ensure feasibility and optimality but are not adaptive and do not support natural language reasoning.

While existing approaches explore the ability of LLMs to act as optimizers, they often lack reliability and struggle with enforcing constraints in complex combinatorial problems. Additionally, purely LLM-based optimization methods do not guarantee feasibility or optimality in structured problems such as TSP.

To address this limitation, this work proposes a hybrid approach where LLMs are used for initial solution generation, and classical optimization techniques are used for refinement and constraint enforcement.

There does not exist any methodology that utilizes both these paradigms together.

3. Problem Statement

Although Generative Artificial Intelligence, especially LLMs, have seen great advances, their practical application towards solving optimization problems with constraints is still not as developed as one would

expect. Although LLMs can be very effective at reasoning and providing answers based on the input text provided to the model, they may still not be able to provide solutions that comply with all the given constraints and optimize certain variables accordingly.

Traditional methods for dealing with optimization problems, which could include implementing the algorithms using Google OR-Tools library, excel in solving constrained problems and always produce feasible solutions but do not have the capability to reason about unstructured input.

Thus, there is a basic incompatibility between generative reasoning models and classical mathematical optimization, each unable to address complex planning problems by itself.

Hence, the research question that will be explored in this paper is:

How to design a unified framework that combines the reasoning capabilities of Large Language Models with the precision and reliability of optimization techniques to effectively solve real-world planning and decision-making problems.

4. Methodology

4.1 Overview

The paper suggests a solution method which is the combination of Large Language Models' reasoning abilities and classical optimization algorithms. It focuses on developing an approach which uses generative AI and constraint-based optimization in cooperation to produce solutions to planning tasks.

The methodology consists of two primary stages:

1. Plan Generation using LLM
2. Plan Optimization using mathematical solvers

4.2 System Architecture

The proposed system follows a structured workflow:

Step 1: Input Processing

- The user provides the problem in natural language
- Example: scheduling, routing, or resource allocation

Step 2: Plan Generation (LLM Layer)

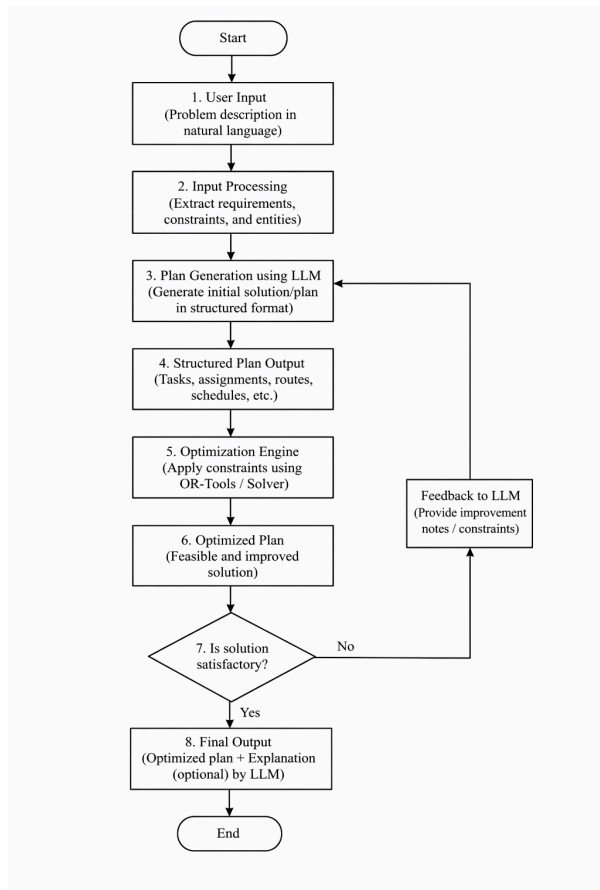
- The LLM interprets the input
- Generates an initial feasible plan
- Produces structured output (tasks, assignments, routes, etc.)

Step 3: Optimization Layer

- The generated plan is passed to an optimization engine
- Constraints such as capacity, cost, and time are applied
- The solution is refined to ensure feasibility and optimality

Step 4: Output Generation

- The optimized solution is returned
- Optional explanation generated by LLM



4.3 Key Components

4.3.1 LLM-Based Planner

- Converts natural language into structured plans
- Handles ambiguity and reasoning
- Generates multiple candidate solutions

4.3.2 Optimization Engine

- Implemented using Google OR-Tools
- Solves constraint-based problems
- Ensures optimal or near-optimal solutions

4.3.3 Integration Layer

- Connects LLM output with optimization input
- Transforms unstructured output into structured format
- Ensures compatibility between both systems

4.4 Workflow Description

The overall workflow can be summarized as:

User Input → LLM Plan Generation → Structured Output → Optimization Engine → Optimized Plan → Final Output

4.5 Proposed Algorithm

1. Accept problem input in natural language
2. Generate initial solution using LLM
3. Convert solution into structured format
4. Apply optimization constraints
5. Solve using optimization engine
6. Return optimized solution
7. Iterate for improvement

4.6 Key Advantages

4.6.1 Performance & Accuracy

Feature	OR-Tools	GenAI (LLM API)
Accuracy	Very high (optimal / near-optimal)	Not guaranteed (heuristic)
Solution Quality	Optimal / mathematically sound	Approximate / suboptimal
Speed	Slower for large-scale problems	Fast response generation
Scalability	Computationally expensive at scale	Easily scales with API usage
Determinism	Deterministic (same output every run)	Non-deterministic outputs

4.6.2 Flexibility & Usability

Feature	OR-Tools	GenAI (LLM API)
Flexibility	Requires structured input	Handles unstructured input
Ease of Use	Requires modeling expertise	Easy via natural language
Adaptability	Needs re-modeling for new problems	Adapts via prompt changes
Human Interaction	Limited	Interactive & conversational
Generalization	Domain-specific	Works across domains

4.6.3 Reliability & Constraints

Feature	OR-Tools	GenAI (LLM API)
Constraint Handling	Strict constraint satisfaction	May violate constraints
Reliability	Highly reliable for optimization	Can produce invalid outputs
Explainability	Limited interpretability	Can explain reasoning
Error Handling	Fails if infeasible	May give incorrect outputs silently
Computation Type	Mathematical optimization	Probabilistic reasoning

5. Experimental Setup

5.1 Problem Definition

The problem considered in this study is the Traveling Salesman Problem (TSP), a classical combinatorial optimization problem. The objective is to determine the shortest possible route that visits a set of cities exactly once and returns to the starting city.

This problem is widely used to evaluate optimization algorithms due to its computational complexity and real-world relevance in logistics, routing, and planning applications.

In this work, the TSP is used as a benchmark to compare the performance of classical optimization techniques and Large Language Model (LLM)-based approaches.

5.2 Dataset

The dataset used in this study is a benchmark Traveling Salesman Problem (TSP) dataset obtained from Kaggle. Each instance in the dataset contains coordinates of multiple cities represented as (x, y) values.

The dataset includes several problem instances with varying numbers of cities, ranging from small-scale (approximately 20–50 cities) to larger-scale instances (over 100 cities). This variation enables evaluation of the scalability and robustness of the proposed approach.

The dataset was preprocessed to extract valid city coordinates from structured columns (City_i_X, City_i_Y) and to construct distance matrices using Euclidean distance. Each instance was treated as an independent TSP problem for experimentation.

This dataset is suitable for evaluating both classical optimization techniques and generative AI-based approaches, as it provides a standardized framework for comparing route optimization performance.

5.3 Prompting Strategy

To evaluate the capability of Large Language Models in solving optimization problems, a structured prompting approach was used.

The LLM was provided with city coordinates in natural language format and instructed to generate a valid route.

Example Prompt:

Solve Traveling Salesman Problem.

Cities:

[(x1, y1), (x2, y2), ...]

Return ONLY the route as a Python list:

[0,1,2,...,0]

DO NOT include any explanation.

This strict prompting format ensures that the model generates structured outputs that can be parsed and evaluated programmatically.

5.4 Implementation

The system was implemented using Python with the following components:

- Pandas for dataset handling

- OR-Tools for solving the TSP using optimization techniques
- OpenAI API for generating routes using Large Language Models

The implementation consists of the following steps:

1. Loading dataset and extracting city coordinates
2. Constructing distance matrix using Euclidean distance
3. Solving TSP using OR-Tools
4. Generating routes using LLM through prompt-based interaction
5. Parsing LLM output and computing total route distance
6. Comparing performance across methods

Example Implementation Snippet:

```
# Extract coordinates
coords = [(x, y) for each city]

# Create distance matrix
distance_matrix = create_distance_matrix(coords)

# Solve using OR-Tools
route = solve_tsp(distance_matrix)

# Calculate total distance
distance = calculate_distance(route, distance_matrix)
```

5.5 Experimental Design

The experiments were designed to compare the performance of two primary approaches:

1. OR-Tools Optimization:
A classical optimization method used to compute optimal or near-optimal solutions.
2. LLM-Based Approach:
A generative method where routes are produced based on natural language prompts.

Due to limitations in handling large inputs, the LLM was evaluated on smaller subsets of cities, while OR-Tools was applied to full problem instances.

Multiple dataset instances were tested to analyze performance across varying problem sizes.

5.6 Execution Environment

The experiments were conducted using Python on a local machine. The implementation is designed to be scalable and can be executed on high-performance computing systems for larger datasets.

The system leverages API-based interaction with LLMs and integrates optimization solvers for efficient computation.

6. Results and Analysis

6.1 Results Table

The performance of the LLM-based approach and OR-Tools optimization was evaluated across multiple instances of the TSP dataset. The results are summarized below.

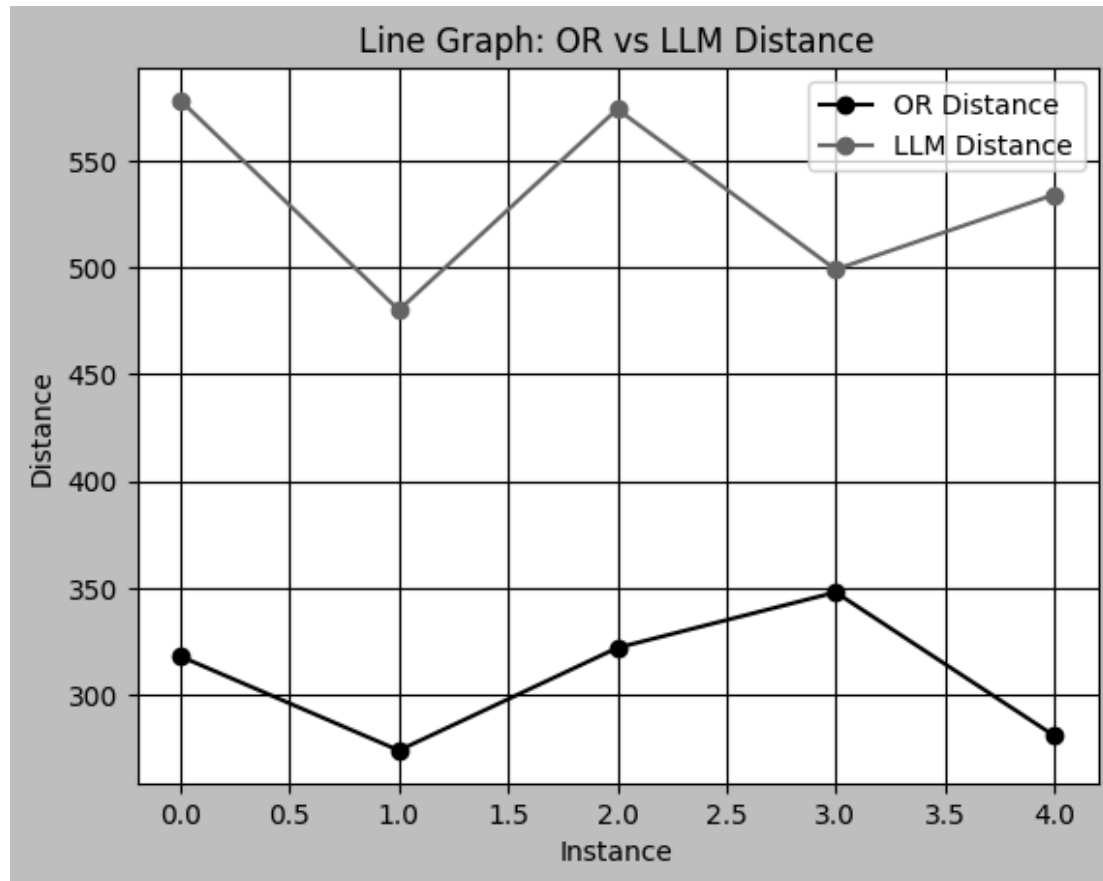
Instance	OR	LLM
0	318	578
1	274	480
2	322	574
3	348	499
4	281	534

6.2 Observations

From the analysis of the experimental results, it can be concluded that the OR-Tools solution method is capable of providing superior performance, in relation to the total distance travelled, compared to the LLM solution method.

LLM-provided tours are feasible yet inferior in nature, with a tendency towards a larger total distance travelled as a result. The reason for this lies in the approximate and heuristic nature of the language models, making them incapable of carrying out exact combinatorial optimization.

Furthermore, an increase in the number of cities causes an even more notable difference in the quality of the results obtained by both methods.



6.3 Analysis

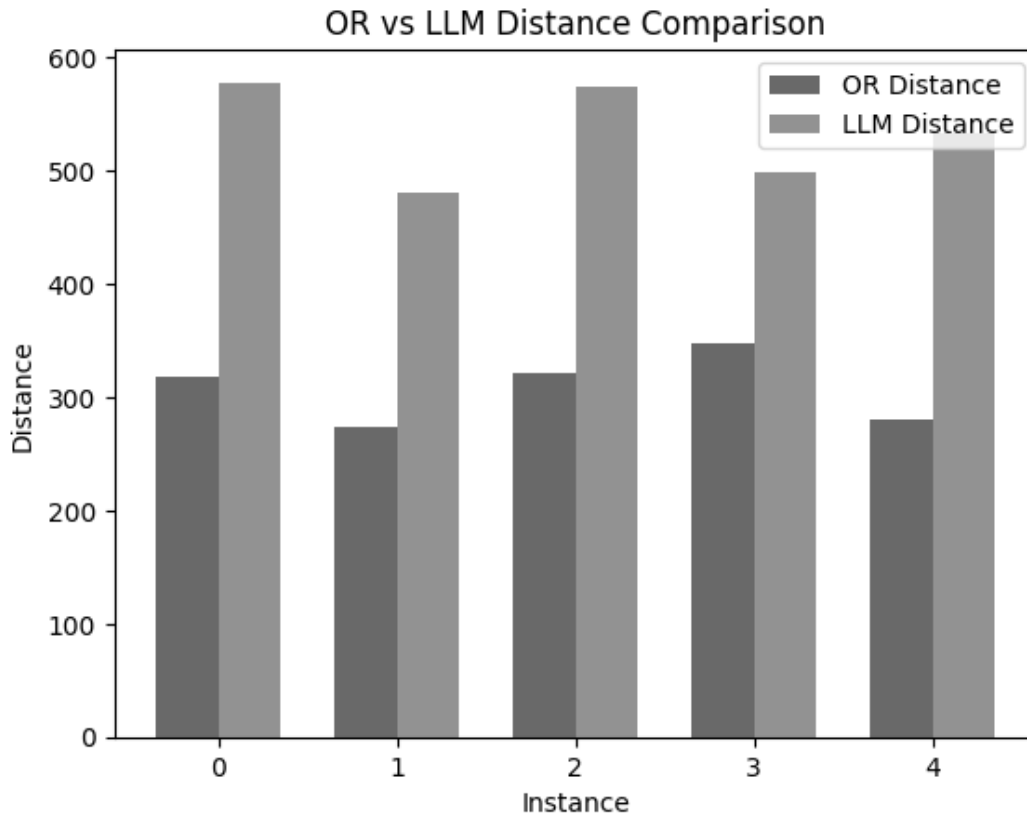
From these findings, it is evident that there is a marked difference between the two optimization techniques and generative AI models.

Being an optimization tool, OR Tools guarantees near-optimal solutions through comprehensive exploration of the search space.

On the other hand, LLMs use learnt patterns and reasoning to output structured solutions; however, these solutions are not optimal and may not be as efficient as those from the former technique.

While having this disadvantage, LLMs have proved effective in learning the structure of the problem and producing viable solutions that are valid routes.

This implies that the hybrid technique of combining the two will be a fruitful course to pursue.



From the graph, it is evident that OR-Tools consistently achieves lower route distances compared to LLM-generated solutions. While LLMs produce feasible routes, their solutions are less optimal due to heuristic reasoning. The gap between the two approaches increases with problem size, reinforcing the limitations of LLMs in solving large-scale optimization problems.

6.4 Key Insights

Key conclusions from the above-mentioned experiments can be summarized as follows:

- Classical optimization techniques solve the problem better than LLMs.
- LLMs provide an efficient method to generate interpretable and feasible solutions.
- LLM efficiency is dependent on the size of the problem; their efficiency decreases with increasing problem size.
- Hybrid models integrating both LLM-based reasoning and optimization can offer a promising solution approach.
- LLMs are more appropriate for unstructured problem-solving than optimization.

8. Conclusion and Future Work

8.1 Conclusion

In this project, a combination of the Large Language Model (LLM) paradigm and classic optimization approaches was implemented to solve the routing problem. The results obtained in solving the Traveling Salesman Problem (TSP) showed that although the LLM paradigm can generate feasible and understandable routes, it cannot produce optimal solutions as a result of its heuristic nature. At the same time, OR-Tools generated substantially shorter routes through careful exploration of the space of possible solutions. Therefore, it is clear that classic optimization approaches continue to outperform LLM approaches in structured and combinatorial problems. Meanwhile, LLM approaches allow for flexible processing of unstructured data inputs and generation of initial solutions.

This study validates the research gap identified in literature, confirming that LLMs alone are insufficient for optimization and require integration with classical methods.

8.2 Future work

Future research could also concentrate on improving the interplay between LLMs and optimization methods in order to facilitate iterative improvement of the solutions. Rather than simply using a one-off solution produced by an LLM, future systems could use methods such as Optimization by Prompting (OPRO), which allows LLMs to iteratively refine their solutions across multiple rounds. Scaling up the methodology to handle larger problem sizes through the use of high-performance computing facilities could also increase the system's practical value. Extending the methodology to tackle more difficult optimization problems in the real world – for example, vehicle routing with capacity limitations, scheduling, and supply chain optimization – could also reveal more information about the capabilities of AI-optimization systems.

9. References

- [1] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large Language Models as Optimizers,” arXiv preprint arXiv:2309.03409, 2023. <https://arxiv.org/abs/2309.03409>
- [2] Google OR-Tools Documentation, “Operations Research Tools by Google,” Available: <https://developers.google.com/optimization>
- [3] Kaggle Dataset, “Traveling Salesman Problem (TSP) Dataset,” <https://www.kaggle.com/datasets/ziya07/traveling-salesman-problem-tsplib-dataset>
- [4] OpenAI API Documentation, “GPT Models and API Usage,” Available: <https://platform.openai.com/docs/>

8. Appendix A: Code Access

The full source code is hosted on GitHub for easy access and reproducibility:

https://github.com/sreeja-moganty/genai_optimization_project.git

